In this case you're mimicking the first way we dealt with FlitServer by by subclassing Flit. The two options shown earlier, singletons and dummy classes, also have analogous usages inside the test.

If you're willing to allow a little bit of manipulation of the original code, you can use Ruby's default arguments to get an almost-seam:

```
def process_flit(flit_server = nil)
  flit_server ||= self.flit_server
  # a bunch of messy stuff
  flit_server.check_for_mentions(self)
  # more messy stuff
  flit_server.follower_list(user)
  # more messy stuff
end
```

In the new method, flit_server is a local that, if not passed as an argument, is given the value of the object's instance method. Thanks to the magic of Ruby's ||= operator, if the argument is passed a value, the passed value is used for the rest of the method. The existing legacy code, which does not use this argument, behaves as is. But it gives you a lever to insert your own server in the test by calling process_flit with the test server as an argument:

```
it "processes a flit correctly if it has followers" do
  user = User.create(screen_name: "zot")
  follower = user.followers.create(screen_name: "jennyw")
  flit = Flit.new(user, "Hello to $jennyw, How are things on earth?")
  flit.process_flit(TestFlitServer.new)
  expect(follower.timeline.size).to eq(1)
end
```

Although this mechanism is slightly intrusive to the original code, you'll probably find you use this pattern often, not just for testing code but also as you add new features to existing code. The default argument lets new code have new behavior while leaving old code behavior untouched.

Each legacy program you work on is going to have its own quirks and require its own kind of creativity, using these methods or others to bring the code the kind of test coverage needed to confidently move forward with bug fixes and new features. As you tackle new problems, remember that reducing dependencies makes it easier to test your code, makes the code cleaner, and makes future work that much easier.

Don't Look Back

It's almost certainly not worth your time and effort to cover an entire complex legacy application before writing any code. I love tests, but the risks involved